Thread Pool

Thread Pool · Cap concurrent thumbnail jobs · Build with Waqar

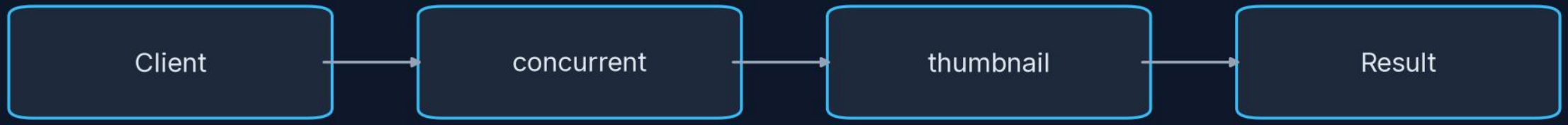
Today's Hook

- Problem: Cap concurrent thumbnail jobs
- Pattern: Thread Pool — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Cap concurrent thumbnail jobs
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Thread Pool: Cap concurrent thumbnail jobs

Real-World Scenario

- Running example: Cap concurrent thumbnail jobs
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.concurrency.threadpool.problem;

import java.util.*;

public final class NaiveThumbs {
  public static int run(int n){ int c=0; for(int i=0;i<n;i++) c++; return c; } // unbounded threads in real life
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.threadpool.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class SimpleThreadPool {
  private final ExecutorService pool;
  SimpleThreadPool(int n){ pool=Executors.newFixedThreadPool(n); }
  <T> Future<T> submit(Callable<T> c){ return pool.submit(c); }
  void shutdown(){ pool.shutdown(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.threadpool.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Thread Pool: cap thumbnail jobs ===");
    System.out.println("--- naive ---"); System.out.println(NaiveThumbs.run(5));
System.out.println("--- thread pool ---");
var pool=new SimpleThreadPool(2); List<Future<Integer>> fs=new ArrayList<>();
for(int i=0;i<5;i++){ int id=i; fs.add(pool.submit(() -> id*id)); }
System.out.println(fs.stream().map(f -> { try { return f.get(); } catch(Exception e){ throw new RuntimeException(e);} }).toList());
pool.shutdown();
  }
}
```

Walkthrough Output

```
=== Thread Pool: cap thumbnail jobs ===
--- naive ---
5
--- thread pool ---
[0, 1, 4, 9, 16]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Thread Pool.

Related Patterns

- Contrast Thread Pool with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Thread Pool → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.